Converting Jupyter Notebooks to PDF

Contents

Overview 1
Option 1: VS Code with the Jupyter Extension 2
  Requirements 2
  Steps 2
  Troubleshooting 2
Option 2: `nbconvert` from the Command Line 3
  Requirements 3
  Steps 4
  Customizing the output 4
Option 3: `nbconvert --to webpdf` (No LaTeX Needed) 4
  Requirements 4
  Steps 4
  Limitations 4
Option 4: Convert to HTML, Then Print to PDF 4
  Steps 4
  Tips 5
Option 5: Google Colab and Other Web-Based Tools 5
  Google Colab 5
  GitHub + nbviewer 5
  Deepnote 5
Option 6: Quarto 5
  Installing LaTeX via Quarto (TinyTeX) 5
  Without LaTeX (Quarto) 6
Quick Decision Guide 6

Overview

If you decide to use a Jupyter Notebook to complete your assignments, you will need to convert it to a PDF for submission to Gradescope. This tutorial covers several ways to do that, from one-click exports to command-line workflows.

The central challenge is that a notebook contains **code outputs** (tables, plots) and **rich text** (Markdown, LaTeX math). Converting both to a single PDF requires a pipeline that can handle each piece. Some methods require LaTeX to be installed; others use your browser's print engine instead.

ℹ Do you need a LaTeX installation?

Method	LaTeX required?
VS Code + Jupyter extension → PDF	Yes
`nbconvert --to pdf`	Yes
`nbconvert --to webpdf`	No
HTML → browser Print to PDF	No
Google Colab → Print	No
Quarto `--to pdf`	Yes

If you don't want to install LaTeX (~4 GB for TeX Live), use one of the **No** options.

Option 1: VS Code with the Jupyter Extension

This is the most convenient if you already use VS Code for notebooks.

Requirements

1. **VS Code** with the Jupyter extension (`ms-toolsai.jupyter`)
2. **Python** with `jupyter` and `nbconvert` installed (`pip install jupyter nbconvert`)
3. **LaTeX** — install one of:
   - **macOS**: MacTeX (~4 GB) or the smaller BasicTeX (~90 MB)
   - **Windows**: MiKTeX
   - **Linux**: `sudo apt install texlive-xetex texlive-latex-extra`

Steps

1. Open your `.ipynb` file in VS Code.
2. Click the **...** (more actions) menu in the top-right of the notebook toolbar.
3. Select **Export** → **PDF**.
4. If this is your first time, VS Code will prompt you to install `nbconvert` if it's missing. Let it do so.
5. If LaTeX is not found, the export will fail — follow the troubleshooting steps below.
6. The PDF will be saved next to your notebook.

Troubleshooting

- **"LaTeX not found"**: VS Code needs to find `pdflatex` or `xelatex` on your system PATH. If the export fails, first verify LaTeX is reachable from the terminal:

  ```
  which pdflatex    # macOS / Linux
  where pdflatex    # Windows (Command Prompt / PowerShell)
  ```

  If that returns a path (e.g., `/Library/TeX/texbin/pdflatex`), LaTeX is installed **but not on the PATH that VS Code sees**. This is the most common issue. How to fix it depends on your OS:

  macOS:

  MacTeX installs to `/Library/TeX/texbin/`. VS Code may not inherit your shell's PATH. Add the TeX binary directory by creating or editing `~/.zshrc` (or `~/.bash_profile`):

```
echo 'export PATH="/Library/TeX/texbin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Then **quit and reopen VS Code** (not just the window — fully quit the application with `Cmd+Q`). VS Code reads the shell environment on launch.

If you use BasicTeX instead of MacTeX, the path is the same (`/Library/TeX/texbin`).

Windows:

MiKTeX typically adds itself to PATH during installation. If `where pdflatex` returns nothing:

1. Find the MiKTeX binary directory (commonly `C:\Users\<username>\AppData\Local\Programs\MiKTeX\miktex\bin\x64\`).
2. Open **Edit environment variables** from the Start menu.
3. Under **System variables** → **Path**, click **Edit** → **New** and paste the directory.
4. **Restart VS Code**.

Linux:

TeX Live installed via `apt` should already be on PATH. If `which pdflatex` returns nothing, install it:

```
sudo apt install texlive-latex-base texlive-latex-extra
```

If you installed TeX Live manually (from the installer script), add it to your shell's rc file:

```
echo 'export PATH="/usr/local/texlive/2024/bin/x86_64-linux:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Then restart VS Code.

Still not working? VS Code's integrated terminal may use a different shell than your system default. Open the VS Code terminal (`Ctrl+\``), run `echo $SHELL`, and make sure your PATH changes are applied to that shell's configuration file.

- **VS Code says nbconvert is missing, but you have it installed**: VS Code may be using a different Python than the one where nbconvert is installed. Check the Python interpreter selected in the VS Code status bar (bottom-right). Switch it to the environment where nbconvert is installed (`Ctrl+Shift+P` → "Python: Select Interpreter").
- **Missing LaTeX packages**: The conversion may fail with errors about missing `.sty` files. Install the missing package through your LaTeX distribution's package manager (`tlmgr install <package>` for TeX Live).
- **Plot rendering issues**: If your plots use special fonts or backends (e.g., PythonPlot in Julia), try re-rendering them with a vector-friendly backend before export.

Option 2: `nbconvert` from the Command Line

If you prefer the terminal or want to automate conversion, use `nbconvert`.

Requirements

- Python with `jupyter` and `nbconvert`: `pip install jupyter nbconvert`
- LaTeX (same as Option 1)

Steps

```
jupyter nbconvert --to pdf notebook.ipynb
```

This creates `notebook.pdf` in the same directory. To keep intermediate files for debugging:

```
jupyter nbconvert --to pdf notebook.ipynb --debug
```

Customizing the output

You can pass a template to control the appearance:

```
jupyter nbconvert --to pdf --template classic notebook.ipynb
```

To hide code cells and show only outputs (useful for reports; **remember, we do not care about your code for grading!**):

```
jupyter nbconvert --to pdf --no-input notebook.ipynb
```

Option 3: `nbconvert --to webpdf` (No LaTeX Needed)

This option uses a headless Chromium browser to render the notebook as a PDF, bypassing LaTeX entirely.

Requirements

- Python with `jupyter` and `nbconvert`: `pip install jupyter nbconvert`
- Playwright for Chromium: `pip install playwright` then `playwright install chromium`

Steps

```
jupyter nbconvert --to webpdf notebook.ipynb
```

The output PDF will closely match what you see when you run `jupyter notebook` in your browser. This is a good option if you do not want to install LaTeX.

Limitations

- Page breaks may be less clean than LaTeX-produced PDFs
- Very large notebooks can be slow to render
- No native support for LaTeX-specific features like equation numbering or cross-references

Option 4: Convert to HTML, Then Print to PDF

This is the **zero-install** method — you only need a browser. It has the same downside as above — page breaks are likely to be messy, and you may need to tweak the print settings.

Steps

1. **Export to HTML** from VS Code (**...** → **Export** → **HTML**) or from the command line:

   ```
   jupyter nbconvert --to html notebook.ipynb
   ```

2. **Open the `.html` file** in Chrome, Firefox, or Safari.
3. **Print to PDF**:

- Chrome/Edge: `Ctrl+P` (Windows) / `Cmd+P` (macOS) → Destination: **Save as PDF**
- Firefox: `Ctrl+P` / `Cmd+P` → **Save to PDF**
- Safari: `Cmd+P` → **PDF** dropdown → **Save as PDF**

4. In the print dialog, enable **Background Graphics** (Chrome) or **Print Backgrounds** (Firefox) so code cell formatting is preserved.

Tips

- Adjust **Margins** to **None** or **Minimum** for a cleaner look.

Option 5: Google Colab and Other Web-Based Tools

Google Colab

If your notebook is in Google Drive or you upload it to Google Colab:

1. Open the notebook in Colab.
2. **File** → **Print** (or `Ctrl+P` / `Cmd+P`).
3. In the print dialog, choose **Save as PDF**.
4. Enable **Background Graphics**.

Colab also has **File** → **Download** → **PDF via LaTeX** (requires a Colab Pro subscription) or **Download as `.ipynb`** (to use one of the other methods).

GitHub + nbviewer

If your notebook is on GitHub:

1. Copy the notebook's GitHub URL.
2. Paste it into nbviewer.org.
3. Click the **Download** icon and select **PDF via LaTeX** (if LaTeX is available on nbviewer's backend) or download as HTML and print.

Deepnote

Deepnote offers a built-in PDF export under **File** → **Export as PDF**.

Option 6: Quarto

If you have Quarto installed[1], it provides the most control over PDF output:

```
quarto render notebook.ipynb --to pdf
```

Quarto uses LaTeX to render PDFs but gives you far more control over formatting. You can add a YAML header to the notebook to customize margins, fonts, and include footnotes and references.

Installing LaTeX via Quarto (TinyTeX)

If you don't have LaTeX installed, Quarto can install a lightweight distribution called **TinyTeX** (~250 MB, much smaller than the full TeX Live):

```
quarto install tinytex
```

[1] Quarto is what I use for all of the course materials, from this website to class notes to assignments (the `*.qmd` files in those repositories are the Quarto source code that notebooks and PDFs are generated from). You can also just use Quarto to write your solution reports directly instead of using a notebook!

This downloads and configures a minimal TeX Live installation scoped to Quarto. After it finishes, `pdflatex` and `xelatex` will be available to Quarto (and to any other tool that looks for them on your PATH, since Quarto adds the TinyTeX binaries to your path).

To update TinyTeX packages later:

```
quarto update tinytex
```

> **Tip**
>
> TinyTeX installs only the packages needed for basic Quarto PDF rendering. If your notebook requires extra LaTeX packages (e.g., `algorithm2e`, `minted`), install them with:
>
> ```
> quarto run tinytex tlmgr install <package-name>
> ```

Without LaTeX (Quarto)

Quarto can also produce a PDF via **Typst**, a modern typesetting engine that is much faster and lighter than LaTeX (~30 MB vs ~4 GB). Typst handles math, tables, and formatting natively — no LaTeX packages needed.

First, install Typst through Quarto:

```
quarto install typst
```

This downloads the Typst compiler and configures Quarto to use it. Once installed, render your notebook:

```
quarto render notebook.ipynb --to typst
```

The output is a `.pdf` with the same filename as your notebook.

Alternatively, Quarto can render via a headless web browser using `wkhtmltopdf`:

```
quarto render notebook.ipynb --to pdf --pdf-engine wkhtmltopdf
```

This requires Chrome or Chromium to be installed but needs neither LaTeX nor Typst.

Quick Decision Guide

You want...	Use
One click in VS Code	Option 1
No LaTeX install, quick result	Option 3 or 4
Automation / scripts	Option 2 or 6
Zero local tools (browser only)	Option 5 (Colab)
Full control over PDF formatting	Option 6 (Quarto)
Notebook-style integration of code and text (possibly without using a Notebook)	Option 6 (Quarto)